

# Uso y necesidad de archivos VHDL

<https://gemini.google.com/app/1b2a236102b9456b>

*User prompt: con esos archivos como podria hacer el ejercicio y que tendria que tener en cuenta para resolverlo , serian necesarios o habria otro metodo mas facil*

Response: Para resolver el ejercicio propuesto en el archivo `image_ca2f39.png`, los códigos VHDL que proporcionaste son herramientas útiles para la etapa de visualización en tu tarjeta de desarrollo, pero no contienen la lógica secuencial central del problema.

## Función de los archivos proporcionados:

- El archivo `primer_comp` funciona como un codificador de prioridad que toma un vector de entrada de 10 bits y lo convierte a un valor binario de 4 bits. En el contexto del proyecto, podrías usarlo para seleccionar o asignar el lugar físico ingresando datos mediante los interruptores de la tarjeta.
- El archivo `segundo_comp` es un decodificador BCD a 7 segmentos que recibe una entrada de 4 bits y genera la salida de 7 bits necesaria para visualizar números en un display. Este bloque será esencial para mostrar el tiempo consumido.

## Elementos que debes tener en cuenta para construir la solución:

- **Divisor de frecuencia:** Es necesario diseñar un módulo que divida el reloj interno de tu placa FPGA para generar una base de tiempo exacta de 1 segundo.
- **Contadores de tiempo:** Debes implementar módulos secuenciales: uno para el temporizador inicial de 35 segundos y otro para medir el tiempo extra de facturación.
- **Máquina de Estados Finitos (FSM):** Requiere lógica comportamental para gestionar los eventos del sistema: detectar el ingreso, iniciar los 35 segundos, encender el LED de alarma si el espacio sigue ocupado, o detener el temporizador y reiniciar el ciclo si la persona sale a tiempo.
- **Integración Estructural:** Siguiendo las sugerencias del ejercicio, debes separar el diseño en casos independientes y luego integrarlos usando una arquitectura estructural. En este módulo principal o *Top-Level* conectarás las salidas de tus contadores a las entradas de instancias de `segundo_comp` para mandar la señal a los displays.

**¿Son necesarios los archivos o existe un método más fácil?** Los archivos proporcionados cubren únicamente la lógica combinacional y no son el sistema completo, pero sí son altamente recomendables para simplificar el proyecto. En el diseño de sistemas digitales con software como Intel Quartus Prime, la metodología más eficiente y "fácil" a largo plazo es exactamente la que sugieren las instrucciones del ejercicio: el diseño modular.

Intentar programar el divisor de reloj, la máquina de estados, los temporizadores y la decodificación de 7 segmentos en un solo bloque gigante genera problemas de síntesis y dificulta la depuración. Por lo tanto, el enfoque correcto es conservar tu archivo `segundo_comp` y usarlo como un componente dentro de un diseño estructural más grande que contenga la lógica de los contadores.

---

*User prompt: hay tutoriales en youtube sobre divisor de frecuencia ?*

Response: Sí, hay excelentes tutoriales en YouTube enfocados específicamente en el diseño de divisores de frecuencia utilizando VHDL, lo cual es ideal para generar la base de tiempo de 1 segundo que necesitas para tu tarjeta de desarrollo.

Aquí tienes algunas de las mejores opciones que te explicarán paso a paso cómo programar este bloque en tu entorno de trabajo:

- **"Divisor de frecuencia en FPGA (Cyclone IV) | Tutorial de lenguaje VHDL"** del canal *D&R Tutoriales*. Este video es muy útil porque se enfoca en la familia de FPGAs de Altera/Intel, lo que facilita su implementación directa en tu software de síntesis.
  - Enlace: <http://www.youtube.com/watch?v=e1t4Shcvyjo>[cite: 4]
- **"Divisor de Frecuencia VHDL primera parte"** del canal *A Electronic*. Es un tutorial muy popular que explica la lógica matemática y secuencial detrás del divisor.
  - Enlace: <http://www.youtube.com/watch?v=n0ja-dEtmsA>[cite: 4]
- **"Divisor de frecuencia VHDL"** del canal *Videotutoriales Eclass*. Un video un poco más extenso y detallado sobre cómo construir el código.
  - Enlace: <http://www.youtube.com/watch?v=edsM5NPh57k>[cite: 4]
- **"VHDL Clase 6: Divisor de Frecuencia"** del canal *Iván González*. Ideal si buscas una explicación estructurada a modo de clase académica.
  - Enlace: <http://www.youtube.com/watch?v=5AwNoAmQBCI>[cite: 4]

Estos videos te mostrarán cómo tomar la señal de reloj de alta velocidad de tu tarjeta (por ejemplo, de 50 MHz) y usar un contador para reducirla hasta obtener un pulso exacto cada segundo, el cual luego podrás conectar a los temporizadores de los 35 segundos de tu proyecto.

[http://googleusercontent.com/youtube\\_content/1](http://googleusercontent.com/youtube_content/1)

*User prompt: explicamelo en quartus*

Response: Para implementar el divisor de frecuencia en Quartus y obtener la base de tiempo de 1 segundo, debes crear un contador en VHDL que cuente los ciclos del reloj principal de tu tarjeta hasta alcanzar el equivalente a un segundo.

Asumiendo que trabajas con una placa como la Altera DE0, el reloj maestro suele ser de 50 MHz (50 millones de ciclos por segundo). Para generar una señal de 1 Hz con un ciclo de trabajo simétrico (50% en alto, 50% en bajo), el contador debe contar hasta 25,000,000 (la mitad de 50 MHz) para invertir el estado de la salida.

Aquí tienes el código VHDL que debes implementar en un nuevo archivo dentro de tu proyecto de Quartus:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity divisor_frecuencia is
  port (
    clk_50Mhz : in std_logic; -- Reloj maestro de la tarjeta (50 MHz)
    reset      : in std_logic; -- Señal para reiniciar el divisor
    clk_1Hz    : out std_logic -- Reloj de salida de 1 segundo
  );
end divisor_frecuencia;
```

```

architecture comportamiento of divisor_frecuencia is
    -- Se usa un rango hasta 24,999,999 (0 a 24,999,999 son 25 millones de ciclos)
    signal contador : integer range 0 to 24999999 := 0;
    signal estado   : std_logic := '0';
begin
    process(clk_50Mhz, reset)
    begin
        if reset = '1' then
            contador <= 0;
            estado <= '0';
        -- Detecta el flanco de subida del reloj de la tarjeta
        elsif rising_edge(clk_50Mhz) then
            if contador = 24999999 then
                estado <= not estado; -- Invierte la señal de salida (toggle)
                contador <= 0;        -- Reinicia el contador para el siguiente ciclo
            else
                contador <= contador + 1; -- Incrementa en cada pulso de 50MHz
            end if;
        end if;
    end process;

    clk_1Hz <= estado; -- Asigna el estado interno a la salida física
end comportamiento;

```

### Pasos para integrarlo en tu proyecto estructural en Quartus:

1. **Agrega el archivo:** Guarda este código en un archivo llamado `divisor_frecuencia.vhd` y compíllalo para verificar que no haya errores de sintaxis.
2. **Conviértelo en un símbolo (Opcional):** Si vas a integrar tus bloques usando el entorno gráfico de Quartus (Block Diagram File), haz clic derecho sobre el archivo en la jerarquía, selecciona **Create/Update** y luego **Create Symbol File for Current File**. Esto te generará un bloque visual que puedes arrastrar a tu diseño principal.
3. **Úsalo como componente en el Top-Level:** Si haces la integración por código, debes declarar este divisor como un componente en la arquitectura de tu archivo principal.
4. **Asignación de Pines (Pin Planner):** En el menú superior de Quartus (Assignments > Pin Planner), debes conectar la entrada `clk_50Mhz` al pin exacto donde la placa recibe su reloj de cristal. La salida `clk_1Hz` no necesita ir a un pin físico externo (como un LED), sino que se convierte en un "cable" interno (signal) que conectarás a la entrada `clk` de los temporizadores de 35 segundos que te pide el ejercicio.

*User prompt: signal contador : integer range 0 to 24999999 := 0; por que explica?*

Response: Esta línea de código se encarga de declarar el contador interno del divisor de frecuencia y está diseñada específicamente para optimizar los recursos físicos de la FPGA.

Aquí tienes el desglose de cada parte y su propósito:

- **signal contador : integer:** Crea una señal (una conexión o registro interno) que almacenará números enteros.

- **range 0 to 24999999:** Esta es la clave del tiempo y del hardware. En una tarjeta como la Altera DE0, el oscilador principal funciona a 50 MHz, lo que significa que emite 50,000,000 de ciclos por segundo.
  - **El cálculo del tiempo:** Para obtener un pulso de 1 Hz que sea simétrico (medio segundo encendido y medio segundo apagado), necesitas que la señal cambie de estado exactamente a la mitad de un segundo. La mitad de 50,000,000 es 25,000,000. Dado que en programación digital el conteo inicia en 0, el ciclo número 25 millones corresponde al valor 24999999.
  - **El ahorro de hardware:** Si solo declaras la señal como integer sin un rango, Quartus Prime asumirá por defecto un tamaño de 32 bits, consumiendo 32 flip-flops en tu tarjeta. Al delimitar el rango máximo hasta 24,999,999, el software de síntesis calcula que solo necesita 25 bits para representar ese número, ahorrando compuertas lógicas y optimizando el diseño.
- **:= 0:** Es el valor de inicialización. Asegura que en el instante en que la tarjeta reciba energía, el contador arranque desde cero antes de empezar a sumar con los flancos del reloj.

*User prompt: si lo coloco 0 to 50000000 ?*

Response: Si lo colocas hasta 50,000,000 y mantienes la lógica del código anterior con estado <= not estado;, tu reloj de salida **no durará 1 segundo, sino 2 segundos** (0.5 Hz).

La razón está en cómo funciona la inversión de la señal (toggle):

#### 1. Con 25,000,000 (el código original):

- El contador cuenta 25 millones de pulsos (0.5 segundos) → la salida pasa a '1'.
- El contador se reinicia y cuenta otros 25 millones de pulsos (0.5 segundos) → la salida pasa a '0'.
- **Total del periodo ( $T$ ):** 0.5 s en alto + 0.5 s en bajo = **1 segundo completo (1 Hz)**.

#### 2. Si colocas 50,000,000 (o 49,999,999):

- Cuenta 50 millones de pulsos → la salida pasa a '1' y permanece encendida durante **1 segundo completo**.
- Cuenta otros 50 millones de pulsos → la salida pasa a '0' y permanece apagada durante **otro segundo completo**.
- **Total del periodo ( $T$ ):** 1 s en alto + 1 s en bajo = **2 segundos**. Tus temporizadores de 35 segundos tardarían en realidad 70 segundos en la vida real.

### ¿Cuándo sí se cuenta hasta 50,000,000 (o 49,999,999)?

Se cuenta hasta 49 999 999 únicamente si **no** inviertes la señal, sino que generas un **pulso de habilitación (enable)** de un solo ciclo cada segundo:

```
if rising_edge(clk_50Mhz) then
  if contador = 49999999 then
    tick_1s <= '1'; -- Genera un pulso por un único ciclo cada 1 segundo
    contador <= 0;
```

```

else
    tick_1s <= '0';
    contador <= contador + 1;
end if;
end if;

```

Esta técnica de generar un pulso de habilitación (*clock enable*) es en realidad la más recomendada en FPGA porque evita crear señales de reloj derivadas que puedan causar desfases (*clock skew*).

*User prompt: me guie de una pagina esto estaria bien para mi proyecto : architecture divisor\_1hz\_arch of divisor\_1hz is signal count : integer range 0 to 50000000 := 0; signal clk\_out : std\_logic := '0'; begin process(clk\_50Mhz, rst) begin if rst = '1' then count <= 0; clk\_out <= '0'; elsif rising\_edge(clk\_50Mhz) then if count = 24999999 then count <= 0; clk\_out <= not clk\_out; else count <= count + 1; end if; end if; end process; clk\_1hz <= clk\_out; end divisor\_1hz\_arch;*

Response: El código funcionará correctamente en la práctica porque la condición de corte dentro del proceso es la correcta:

- Al evaluar `if count = 24999999 then`, la variable se reinicia a 0 cada 25 000 000 de ciclos y alterna el valor de `clk_out` (`not clk_out`), generando con precisión la onda cuadrada de 1 Hz (1 segundo de periodo total).
- La señal `clk_out` nunca alcanzará los valores superiores a 24 999 999.

Sin embargo, hay una inconsistencia técnica en la declaración del rango:

- Tienes declarado `signal count : integer range 0 to 50000000 := 0;`.
- Como tu condición de reinicio está en 24 999 999, declarar el rango hasta 50 000 000 reserva espacio innecesario para bits que el contador jamás utilizará (50 000 000 requiere 26 bits en hardware, mientras que 24 999 999 solo necesita 25 bits).

Para dejar el código limpio y optimizado para la síntesis en Quartus, ajusta la declaración de la señal a:

```
signal count : integer range 0 to 24999999 := 0;
```

Con ese cambio menor, el módulo es totalmente adecuado para servir como base de tiempo de 1 segundo en tu proyecto.

*User prompt: ejemplos de contadores en la web*

Response: En la literatura y sitios web de diseño digital en VHDL (como *Nangate*, *FPGA4student*, *Nandland* o la documentación de Intel FPGA), los contadores se clasifican según su arquitectura y función. A continuación se presentan los tres modelos más comunes y representativos que se adaptan a tu proyecto.

## 1. Contador Binario Parametrizable con Habilitador (*Enable*)

Es el ejemplo clásico encontrado en tutoriales web para controlar eventos por ciclos de reloj. Permite pausar o activar la cuenta mediante una señal de control (*ena*).

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity contador_binario is
    generic (
        VALOR_MAX : integer := 35
    );
    port (
        clk      : in  std_logic;
        rst      : in  std_logic;
        ena      : in  std_logic;
        fin      : out std_logic;
        cuenta   : out std_logic_vector(5 downto 0)
    );
end contador_binario;

architecture behavioral of contador_binario is
    signal reg_cuenta : integer range 0 to VALOR_MAX := 0;
begin
    process(clk, rst)
    begin
        if rst = '1' then
            reg_cuenta <= 0;
            fin <= '0';
        elsif rising_edge(clk) then
            if ena = '1' then
                if reg_cuenta = VALOR_MAX then
                    reg_cuenta <= 0;
                    fin <= '1'; -- Genera un flag al alcanzar el valor límite
                else
                    reg_cuenta <= reg_cuenta + 1;
                    fin <= '0';
                end if;
            end if;
        end if;
    end process;

    cuenta <= std_logic_vector(to_unsigned(reg_cuenta, 6));
end behavioral;

```

## 2. Contador BCD de 2 Dígitos (Decenas y Unidades)

En proyectos de visualización en displays de 7 segmentos (como el tuyo con `segundo_comp.vhd`), la web recomienda ampliamente los contadores en formato BCD (Binary-Coded Decimal). En lugar de contar directamente en binario hasta 35, cuenta unidades (0 a 9) y decenas (0 a 3 o más), lo que facilita enviar cada dígito directamente a tu decodificador sin requerir operaciones de división en hardware.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity contador_bcd_2digitos is
    port (

```

```

        clk      : in  std_logic;          -- Conectado a la salida clk_1hz
        rst      : in  std_logic;
        ena      : in  std_logic;
        unidades : out std_logic_vector(3 downto 0); -- Al display de unidades
        decenas  : out std_logic_vector(3 downto 0); -- Al display de decenas
    );
end contador_bcd_2digitos;

architecture rtl of contador_bcd_2digitos is
    signal u : integer range 0 to 9 := 0;
    signal d : integer range 0 to 9 := 0;
begin
    process(clk, rst)
    begin
        if rst = '1' then
            u <= 0;
            d <= 0;
        elsif rising_edge(clk) then
            if ena = '1' then
                if u = 9 then
                    u <= 0;
                    if d = 9 then
                        d <= 0;
                    else
                        d <= d + 1;
                    end if;
                else
                    u <= u + 1;
                end if;
            end if;
        end if;
    end process;

    unidades <= std_logic_vector(to_unsigned(u, 4));
    decenas  <= std_logic_vector(to_unsigned(d, 4));
end rtl;

```

### 3. Contador Descendente con Parada Automática (*Timer / Timeout*)

Este patrón se utiliza frecuentemente en foros y repositorios para temporizadores de cuenta regresiva (como esperar exactamente 35 segundos y detenerse al llegar a 0).

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity temporizador_descendente is
    port (
        clk      : in  std_logic;
        carga    : in  std_logic; -- Carga el valor inicial (ej. 35)
        alarma   : out std_logic; -- Se activa al llegar a 0
        seg_out  : out integer range 0 to 35
    );
end temporizador_descendente;

```

```
architecture behavioral of temporizador_descendente is
    signal timer : integer range 0 to 35 := 35;
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if carga = '1' then
                timer <= 35;
                alarma <= '0';
            elsif timer > 0 then
                timer <= timer - 1;
                alarma <= '0';
            else
                alarma <= '1'; -- Tiempo terminado
            end if;
        end if;
    end process;

    seg_out <= timer;
end behavioral;
```

Comparación para tu ejercicio

| Tipo de Contador       | Ventaja principal                                                         | Adecuación para tu proyecto                                  |
|------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------|
| Binario Simple         | Menor cantidad de lógica y fácil de comparar (timer = 35).                | Requiere convertir a BCD antes de mandar a segundo_comp.vhd. |
| BCD (Unidades/Decenas) | Conexión directa a las entradas de 4 bits de tus displays de 7 segmentos. | Ideal para mostrar el tiempo en dos dígitos de la tarjeta.   |
| Descendente            | Detecta el fin del ciclo naturalmente cuando el registro alcanza 0.       | Excelente si deseas mostrar una cuenta regresiva de 35 a 0.  |

¿Prefieres que el temporizador de 35 segundos muestre el tiempo de forma ascendente (de 0 a 35) o regresiva (de 35 a 0)?

```
User prompt: mira este , esta bien o lo puedo corregir library IEEE; use IEEE.STD_LOGIC_1164.ALL; entity control_parqueo is port ( clk_1hz : in std_logic; sensor : in std_logic; rst : in std_logic; contador_35unidades : out integer range 0 to 9; contador_35decenas : out integer range 0 to 9; contador_extraunidades : out integer range 0 to 9; contador_extradecenas : out integer range 0 to 9; alarma : out std_logic; felicitacion : out std_logic ); end control_parqueo; architecture comportamental of control_parqueo is signal tiempo_35s : integer range 0 to 35 := 0; signal tiempo_extra : integer range 0 to 99 := 0; type tipo_estado is (LIBRE, CONTANDO_35, EXTRA_TIEMPO, FIN_FELIZ); signal estado_actual : tipo_estado := LIBRE; begin process(clk_1hz, rst) begin if rst = '1' then estado_actual <= LIBRE; tiempo_35s <= 0; tiempo_extra <= 0; alarma <= '0'; felicitacion <= '0'; elsif rising_edge(clk_1hz) then case estado_actual is when LIBRE => tiempo_35s <= 0; tiempo_extra <= 0; alarma <= '0'; felicitacion <= '0'; if sensor = '1' then estado_actual <= CONTANDO_35; end if; when CONTANDO_35 => if sensor = '0' then estado_actual <= FIN_FELIZ;
```

```

    felicitacion <= '1';           else           if tiempo_35s < 35 then           tiempo_35s
    <= tiempo_35s + 1;           else           estado_actual <= EXTRA_TIEMPO;
    alarma <= '1';           end if;           end if;           when EXTRA_TIEMPO
=>           if sensor = '0' then           estado_actual <= LIBRE;           else
           if tiempo_extra < 99 then           tiempo_extra <= tiempo_extra + 1;
end if;           end if;           when FIN_FELIZ =>           if sensor = '1' then
           estado_actual <= CONTANDO_35;           felicitacion <= '0';           tiempo_35s
<= 0;           end if;           end case;           end if;           end process;           contador_35unidades <=
tiempo_35s mod 10;           contador_35decenas <= tiempo_35s / 10;           contador_extraunidades <=
tiempo_extra mod 10;           contador_extradecenas <= tiempo_extra / 10; end comportamental;

```

Response: El código está muy bien planteado en cuanto a la lógica de la máquina de estados (FSM): cubre los casos solicitados, transiciona correctamente según el sensor y maneja la alarma y la felicitación.

Sin embargo, tiene **3 detalles críticos** que debes corregir para que sintetice sin problemas en Quartus y se conecte a tus otros componentes:

## Puntos a corregir

### 1. Uso de / 10 y mod 10 en síntesis:

- En VHDL sintetizable para FPGAs, la división (/) y el módulo (mod) entre números que no son potencias de 2 (como 10) consumen bastantes recursos lógicos o a veces generan advertencias/errores en Quartus si no se incluye `IEEE.NUMERIC_STD.ALL`.
- En su lugar, es más limpio llevar contadores BCD separados (unidades que van de 0 a 9 y decenas que se incrementan cuando las unidades llegan a 9) o asegurar el uso de las bibliotecas correspondientes.

### 2. Compatibilidad de tipos de puerto con `segundo_comp.vhd`:

- Tu archivo `segundo_comp.vhd` espera una entrada `std_logic_vector(3 downto 0)`.
- En tu entidad declaraste las salidas como `integer range 0 to 9`.
- Para conectarlos directamente en un archivo estructural sin hacer conversiones de tipo en el *top-level*, es mucho mejor que las salidas sean directamente `std_logic_vector(3 downto 0)`.

### 3. Reinicio de la felicitación y la alarma:

- Cuando pasas de `EXTRA_TIEMPO` a `LIBRE`, no estás apagando explícitamente `alarma <= '0'`; en esa transición (aunque en `LIBRE` se pone en `'0'`, tarda un ciclo adicional en verse reflejado).
- Al pasar a `FIN_FELIZ`, la felicitación queda encendida indefinidamente hasta que entra otra persona. Si la persona sale, usualmente la felicitación se muestra durante unos segundos o se apaga al quedar libre. Si el enunciado pide "esperar hasta que una persona ingrese nuevamente", tu lógica en `FIN_FELIZ` cumple con el texto, pero es buena práctica garantizar que los contadores se limpien ahí.

## Código corregido y optimizado para Quartus

Aquí tienes la versión ajustada utilizando `std_logic_vector` de 4 bits (listo para conectar a `segundo_comp.vhd`) y convirtiendo los enteros con `to_unsigned` de la biblioteca estándar `IEEE.NUMERIC_STD`:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity control_parqueo is
    port (
        clk_1hz          : in  std_logic;
        sensor            : in  std_logic;
        rst               : in  std_logic;
        contador_35unidades : out std_logic_vector(3 downto 0);
        contador_35decenas  : out std_logic_vector(3 downto 0);
        contador_extraunidades : out std_logic_vector(3 downto 0);
        contador_extradecenas : out std_logic_vector(3 downto 0);
        alarma             : out std_logic;
        felicitacion        : out std_logic
    );
end control_parqueo;

architecture comportamental of control_parqueo is
    signal tiempo_35s : integer range 0 to 35 := 0;
    signal tiempo_extra : integer range 0 to 99 := 0;

    type tipo_estado is (LIBRE, CONTANDO_35, EXTRA_TIEMPO, FIN_FELIZ);
    signal estado_actual : tipo_estado := LIBRE;
begin

    process(clk_1hz, rst)
    begin
        if rst = '1' then
            estado_actual <= LIBRE;
            tiempo_35s    <= 0;
            tiempo_extra  <= 0;
            alarma        <= '0';
            felicitacion  <= '0';
        elsif rising_edge(clk_1hz) then
            case estado_actual is

                when LIBRE =>
                    tiempo_35s    <= 0;
                    tiempo_extra  <= 0;
                    alarma        <= '0';
                    felicitacion  <= '0';
                    if sensor = '1' then
                        estado_actual <= CONTANDO_35;
                    end if;

                when CONTANDO_35 =>
                    if sensor = '0' then
                        estado_actual <= FIN_FELIZ;
                        felicitacion  <= '1';
                    else
                        if tiempo_35s < 35 then
                            tiempo_35s <= tiempo_35s + 1;
                        else

```

```

        estado_actual <= EXTRA_TIEMPO;
        alarma        <= '1';
    end if;
end if;

when EXTRA_TIEMPO =>
    if sensor = '0' then
        alarma        <= '0';
        estado_actual <= LIBRE;
    else
        if tiempo_extra < 99 then
            tiempo_extra <= tiempo_extra + 1;
        end if;
    end if;

when FIN_FELIZ =>
    -- Espera hasta que ingrese una nueva persona
    if sensor = '1' then
        tiempo_35s     <= 0;
        felicitacion   <= '0';
        estado_actual <= CONTANDO_35;
    end if;

end case;
end if;
end process;

-- Descomposición de decenas y unidades convertidas a vectores de 4 bits
contador_35unidades    <= std_logic_vector(to_unsigned(tiempo_35s mod 10, 4));
contador_35decenas     <= std_logic_vector(to_unsigned(tiempo_35s / 10, 4));
contador_extraunidades <= std_logic_vector(to_unsigned(tiempo_extra mod 10, 4));
contador_extradecenas  <= std_logic_vector(to_unsigned(tiempo_extra / 10, 4));

end comportamental;

```

Con estas salidas en `std_logic_vector(3 downto 0)`, puedes conectar directamente cada una a la entrada `g` del componente `segundo_comp.vhd` para ver las decenas y unidades en los displays de 7 segmentos de la tarjeta.

---

*User prompt: pero si tengo este archivo ya podria continuar al top level ? library IEEE; use IEEE.STD\_LOGIC\_1164.ALL; entity display\_driver is port ( val\_in : in integer range 0 to 9; seg\_out : out std\_logic\_vector(6 downto 0) ); end display\_driver; architecture estructural of display\_driver is component primer\_comp is port ( k : in integer range 0 to 9; d : out std\_logic\_vector(3 downto 0)); end component; component segundo\_comp is port ( g : in std\_logic\_vector(3 downto 0); f : out std\_logic\_vector(6 downto 0)); end component; signal cable\_bcd : std\_logic\_vector(3 downto 0); begin U1: primer\_comp port map (k => val\_in, d => cable\_bcd); U2: segundo\_comp port map (g => cable\_bcd, f => seg\_out); end estructural;*

Response: Todavía **no** puedes pasar al *Top-Level* tal como está, porque hay un conflicto grave de compatibilidad con `primer_comp.vhd`:

## El error en `display_driver`

En el archivo original que compartiste al inicio, la entidad de `primer_comp` está definida así:

```
entity primer_comp is port (
    k : in std_logic_vector(9 downto 0); -- << Es un vector de 10 bits tipo "one-hot"
    d : out std_logic_vector(3 downto 0));
end primer_comp;
```[cite: 1]
```

Sin embargo, en tu `display_driver` declaraste el componente así:

```
```vhdl
component primer_comp is
    port ( k : in integer range 0 to 9; ... ); -- << ERROR DE TIPOS
end component;
```

Quartus arrojará un error de síntesis (*Type mismatch / Port mismatch*) al intentar compilar porque los puertos del componente no coinciden con la entidad real de `primer_comp`.

## ¿Realmente necesitas `primer_comp` para los displays?

**No.** `primer_comp` no es un convertidor de enteros a BCD, sino un codificador que recibe 10 líneas de entrada (como 10 interruptores individuales donde solo se activa uno a la vez).

Para mostrar los números de los contadores en los displays de 7 segmentos:

1. Si tus contadores entregan `std_logic_vector(3 downto 0)` (BCD de 0 a 9), **solo necesitas conectar directamente `segundo_comp`** a cada display.
2. Si prefieres que tu módulo de control entregue `integer range 0 to 9` y quieres un módulo intermediario `display_driver`, este no necesita a `primer_comp`; solo debe convertir el entero a `std_logic_vector(3 downto 0)` con `to_unsigned` y pasárselo a `segundo_comp`:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity display_driver is
    port (
        val_in  : in integer range 0 to 9;
        seg_out : out std_logic_vector(6 downto 0)
    );
end display_driver;

architecture estructural of display_driver is
    component segundo_comp is
        port (
            g : in std_logic_vector(3 downto 0);
            f : out std_logic_vector(6 downto 0)
        );
    end component;

    signal cable_bcd : std_logic_vector(3 downto 0);
begin
    cable_bcd <= std_logic_vector(to_unsigned(val_in, 4));

    U2: segundo_comp port map (
```

```

        g => cable_bcd,
        f => seg_out
    );
end estructural;
```

Con esa corrección (o descartando `display_driver` y conectando directamente `segundo_comp` si tus contadores ya entregan 4 bits), el proyecto queda listo para construir el *Top-Level*.

---

*User prompt: esto en que afecta si quiero que en mi display se pueda colocar el contador en los dos primeros displays de los 35s y en los otros dos los segundos extra*

Response: Tener esa arquitectura afecta positivamente porque hace la distribución de hardware **mucho más simple y directa**: al contar con 4 displays físicos (HEX0, HEX1, HEX2, HEX3 en tu tarjeta), no requieres circuitos de multiplexación de pantalla ni switches para alternar lecturas.

Así afecta la estructura del proyecto y lo que debes tener en cuenta:

## 1. Cuántas instancias de decodificador necesitas

Como cada display de 7 segmentos requiere su propio bus de 7 pines (`std_logic_vector(6 downto 0)`), necesitas **4 instancias independientes de decodificador**:

- **HEX0**: Unidades del tiempo de 35 s.
- **HEX1**: Decenas del tiempo de 35 s.
- **HEX2**: Unidades del tiempo extra.
- **HEX3**: Decenas del tiempo extra.

## 2. Dos formas de implementarlo en el Top-Level

### Opción A: Usando `display_driver` (si mantienes las salidas como integer range 0 to 9)

Si tu módulo `control_parqueo` conserva las 4 salidas como enteros de 0 a 9, en el *Top-Level* simplemente declaras el componente `display_driver` (la versión corregida) y creas 4 copias:

```

-- Muestra el tiempo de 35s
DISP_35_U : display_driver port map(val_in => cable_35_u,    seg_out => HEX0);
DISP_35_D : display_driver port map(val_in => cable_35_d,    seg_out => HEX1);

-- Muestra el tiempo extra
DISP_EX_U : display_driver port map(val_in => cable_extra_u, seg_out => HEX2);
DISP_EX_D : display_driver port map(val_in => cable_extra_d, seg_out => HEX3);
```

### Opción B: Usando `segundo_comp` directamente (la vía más limpia)

Si en tu `control_parqueo` las salidas son `std_logic_vector(3 downto 0)`, no necesitas `display_driver`. Conectas 4 instancias de `segundo_comp` directamente en el *Top-Level*:

```

DISP_35_U : segundo_comp port map(g => bcd_35_u,    f => HEX0);[cite: 2]
DISP_35_D : segundo_comp port map(g => bcd_35_d,    f => HEX1);[cite: 2]
```

```
DISP_EX_U : segundo_comp port map(g => bcd_extra_u, f => HEX2);[cite: 2]
DISP_EX_D : segundo_comp port map(g => bcd_extra_d, f => HEX3);[cite: 2]
```

### 3. Comportamiento visual en la tarjeta

Debes considerar qué mostrarán los displays HEX2 y HEX3 durante los primeros 35 segundos:

- Con el código actual, tiempo\_extra permanece en 0 hasta que se cumplen los 35 segundos.
- Por lo tanto, los dos displays de la izquierda mostrarán 00 mientras los dos de la derecha avanzan de 00 a 35.
- Una vez transcurridos los 35 segundos, HEX0 y HEX1 se congelarán en 35 (o se apagan según decidas) y HEX2 y HEX3 empezarán a incrementar (01, 02, etc.).

*User prompt: entonces que archivos debo utilizar par el top level ?*

*Response:* Para construir y sintetizar el archivo principal (*Top-Level*) en Quartus sin redundancias ni errores de tipos, debes utilizar únicamente **3 archivos componentes** además de tu propio archivo *Top-Level*:

- **divisor\_1hz.vhd:** Genera la base de tiempo de 1 Hz a partir del oscilador maestro de 50 MHz.
- **control\_parqueo.vhd:** Maneja la máquina de estados (detección de ingreso, temporizador de 35 segundos, conteo de tiempo extra, alarma y felicitación) entregando los dígitos de unidades y decenas en formato binario de 4 bits (std\_logic\_vector(3 downto 0)).
- **segundo\_comp.vhd:** Decodifica cada valor de 4 bits al bus de 7 segmentos para los displays de la tarjeta. En el *Top-Level* simplemente instancias este componente 4 veces (dos para los 35 s y dos para el tiempo extra).

### ¿Qué pasa con los otros archivos?

- **primer\_comp.vhd:** **No se utiliza.** Es un codificador de prioridad de 10 bits a 4 bits que no interviene en la ruta entre los contadores de tiempo y los displays.
- **display\_driver.vhd:** **Tampoco es necesario.** Resulta redundante porque conectar cuatro instancias de segundo\_comp directamente en el *Top-Level* simplifica la jerarquía del diseño y evita errores de conversión.

### Esquema del archivo Top-Level (sistema\_parqueo\_top.vhd)

Este es el archivo principal que debes crear en Quartus para enlazar los 3 módulos:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sistema_parqueo_top is
    port (
        CLOCK_50      : in  std_logic;           -- Reloj de 50 MHz de la placa
        SW_RESET       : in  std_logic;           -- Pulsador/Switch de reset
        SW_SENSOR       : in  std_logic;           -- Switch/Sensor de presencia
        LED_ALARMA      : out std_logic;           -- Alarma de tiempo cumplido
        LED_FELIZ       : out std_logic;           -- Felicitación
    );
end entity;
```

```

    HEX0      : out std_logic_vector(6 downto 0); -- 35s: Unidades
    HEX1      : out std_logic_vector(6 downto 0); -- 35s: Decenas
    HEX2      : out std_logic_vector(6 downto 0); -- Extra: Unidades
    HEX3      : out std_logic_vector(6 downto 0)  -- Extra: Decenas
  );
end sistema_parqueo_top;

```

architecture estructural of sistema\_parqueo\_top is

```

-- 1. Declaración del divisor de frecuencia
component divisor_1hz is

```

```

  port (
    clk_50Mhz : in  std_logic;
    rst       : in  std_logic;
    clk_1hz   : out std_logic
  );
end component;

```

```

-- 2. Declaración del módulo de control
component control_parqueo is

```

```

  port (
    clk_1hz           : in  std_logic;
    sensor            : in  std_logic;
    rst               : in  std_logic;
    contador_35unidades : out std_logic_vector(3 downto 0);
    contador_35decenas  : out std_logic_vector(3 downto 0);
    contador_extraunidades : out std_logic_vector(3 downto 0);
    contador_extradecenas : out std_logic_vector(3 downto 0);
    alarma            : out std_logic;
    felicitacion       : out std_logic
  );
end component;

```

```

-- 3. Declaración del decodificador de 7 segmentos
component segundo_comp is

```

```

  port (
    g : in  std_logic_vector(3 downto 0);
    f : out std_logic_vector(6 downto 0)
  );
end component;

```

```

-- Señales internas de interconexión

```

```

signal cable_1hz      : std_logic;
signal bcd_35_u       : std_logic_vector(3 downto 0);
signal bcd_35_d       : std_logic_vector(3 downto 0);
signal bcd_extra_u    : std_logic_vector(3 downto 0);
signal bcd_extra_d    : std_logic_vector(3 downto 0);

```

begin

```

-- Instancia de la base de tiempo
U_DIV : divisor_1hz
  port map (
    clk_50Mhz => CLOCK_50,

```

```

        rst      => SW_RESET,
        clk_1hz   => cable_1hz
    );

-- Instancia de la lógica de control y temporizadores
U_CTRL : control_parqueo
    port map (
        clk_1hz      => cable_1hz,
        sensor        => SW_SENSOR,
        rst           => SW_RESET,
        contador_35unidades => bcd_35_u,
        contador_35decenas  => bcd_35_d,
        contador_extraunidades => bcd_extra_u,
        contador_extradecenas => bcd_extra_d,
        alarma         => LED_ALARMA,
        felicitacion     => LED_FELIZ
    );

-- Instancias para los 4 displays de 7 segmentos
DISP_35_UNIDADES : segundo_comp
    port map (g => bcd_35_u, f => HEX0);

DISP_35_DECENAS : segundo_comp
    port map (g => bcd_35_d, f => HEX1);

DISP_EXTRA_UNIDADES : segundo_comp
    port map (g => bcd_extra_u, f => HEX2);

DISP_EXTRA_DECENAS : segundo_comp
    port map (g => bcd_extra_d, f => HEX3);

end estructural;

```

---

*User prompt: afecta si uno a los archivos de divisor, control parqueo y segundo componente ?*

Response: Unir todos los códigos en un solo archivo VHDL afecta directamente dos aspectos: la evaluación académica del ejercicio y el flujo de trabajo en Quartus.

## 1. Afecta los requisitos de la guía

En la lista de sugerencias de la imagen se solicita explícitamente:

- *"Integrar los dos casos después de validarlos".*
- *"Usar diferentes arquitecturas (flujo de datos, estructural y comportamental)".*

Si metes todo en un único archivo con un solo proceso gigante, pierdes la **arquitectura estructural** (que es justamente la interconexión de componentes mediante señales internas y `port map`). Mantener los módulos separados o interconectados estructuralmente demuestra que sabes modularizar hardware.

## 2. Formas de tenerlos en "un solo archivo" sin perder la arquitectura estructural

Si te refieres a pegar el código de las entidades una detrás de otra dentro de un mismo archivo .vhd en Quartus, **sí se puede compilar**, pero debes tener cuidado con lo siguiente:

- **Múltiples entidades en un .vhd:** Puedes escribir en el mismo archivo la entidad y arquitectura de divisor\_1hz, luego las de control\_parqueo, las de segundo\_comp y al final la de sistema\_parqueo\_top.
- **Top-Level Entity en Quartus:** En Quartus debes asegurarte de definir sistema\_parqueo\_top como la entidad principal del proyecto (**Project > Set as Top-Level Entity**). Si el archivo tiene el nombre de otro componente, Quartus podría intentar sintetizar ese submódulo por error en lugar del circuito completo.

¿Es recomendable unirlos?

- **No es recomendable para depurar:** Si algo falla en la lógica de conteo o en el reloj, probar un archivo unificado es más complejo que probar y simular cada componente por separado.
- **El estándar en diseño digital es mantener un archivo por componente:** Un archivo divisor\_1hz.vhd, un archivo control\_parqueo.vhd, un archivo segundo\_comp.vhd y un archivo sistema\_parqueo\_top.vhd. Quartus los compila todos automáticamente si están añadidos al proyecto.

User prompt: como funcionaria el top levele que me diste en la fpga?

Response: En la tarjeta FPGA física, el archivo Top-Level actúa como la central de conexiones que toma las entradas de los periféricos mecánicos (switches y pulsadores), procesa los datos mediante los componentes internos y muestra los resultados en los actuadores visuales (LEDs y displays de 7 segmentos).

Distribución de hardware en la FPGA

| Elemento físico        | Puerto asignado en VHDL | Función en el sistema                                                              |
|------------------------|-------------------------|------------------------------------------------------------------------------------|
| Oscilador de 50 MHz    | CLOCK_50                | Alimenta al divisor para generar el pulso de 1 segundo internamente.               |
| Interruptor (Switch 0) | SW_SENSOR               | Simula la presencia de la persona en el espacio (arriba = ocupado, abajo = libre). |
| Pulsador/Switch 1      | SW_RESET                | Reinicia todo el circuito a las condiciones iniciales.                             |
| LED Verde              | LED_FELIZ               | Enciende si la persona sale a tiempo ("felicitación").                             |
| LED Rojo               | LED_ALARMA              | Enciende si se superan los 35 segundos ("alarma").                                 |
| Displays HEX1 - HEX0   | HEX1 y HEX0             | Muestran las decenas y unidades del temporizador base de 35 segundos.              |
| Displays HEX3 - HEX2   | HEX3 y HEX2             | Muestran las decenas y unidades del temporizador de tiempo extra.                  |

## Secuencia de funcionamiento paso a paso

### 1. Estado Inicial (Reposo / Espacio Libre):

- El switch SW\_SENSOR está en '0'.
- Los displays muestran 00 en el tiempo extra (HEX3-HEX2) y 00 en el tiempo inicial (HEX1-HEX0).
- Ambos LEDs (LED\_ALARMA y LED\_FELIZ) permanecen apagados.

### 2. Ingreso de la persona:

- Mueves SW\_SENSOR a '1'.
- El divisor\_1hz empieza a entregar pulsos de 1 segundo al bloque de control.
- Los displays HEX1 y HEX0 comienzan a contar hacia arriba segundo a segundo (01, 02, ..., 34, 35). Los displays HEX3-HEX2 continúan en 00.

### 3. Caso A — La persona sale a tiempo (antes o al llegar a 35 s):

- Bajas SW\_SENSOR a '0' mientras el conteo va, por ejemplo, en 20.
- El conteo se detiene de inmediato.
- Se enciende el LED\_FELIZ indicando la felicitación.
- El sistema queda a la espera hasta que SW\_SENSOR vuelva a subir a '1', momento en el que el LED se apaga y el contador se reinicia.

### 4. Caso B — La persona excede el tiempo asignado:

- El switch SW\_SENSOR se mantiene en '1' y HEX1-HEX0 alcanzan el valor 35.
- En ese instante se enciende el LED\_ALARMA.
- HEX1-HEX0 se detienen marcando 35 y los displays de la izquierda (HEX3-HEX2) comienzan a incrementar segundo a segundo (01, 02, 03...), registrando el cobro extra.
- Cuando la persona finalmente se retira (bajas SW\_SENSOR a '0'), la alarma se apaga y el sistema regresa a su estado libre inicial.

*User prompt: así me quedo todo , funcionaria bien?*

Response: El archivo `sistema_de_parqueo_1.vhd` declara el componente `control_parqueo` requiriendo que sus cuatro salidas de conteo sean de tipo `std_logic_vector(3 downto 0)`. En contraste, el código original de la entidad `control_parqueo.vhd` define estas mismas salidas como `integer range 0 to 9`. Al momento de sintetizar el circuito, el compilador detectará que los puertos declarados en el *Top-Level* no coinciden con los puertos reales de la entidad de control, generando un error de incompatibilidad de tipos (Port Mismatch). Por lo tanto, el sistema no funcionará en su estado actual.

Para corregir este error, debes modificar exclusivamente el archivo `control_parqueo.vhd` para que realice la conversión de enteros a vectores de 4 bits, permitiendo su conexión directa con los puertos del archivo `sistema_de_parqueo_1.vhd` y posteriormente con los decodificadores `segundo_comp`.

Reemplaza el código de tu archivo control\_parqueo.vhd por el siguiente:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity control_parqueo is
    port (
        clk_1hz          : in std_logic;
        sensor            : in std_logic;
        rst               : in std_logic;
        contador_35unidades : out std_logic_vector(3 downto 0);
        contador_35decenas  : out std_logic_vector(3 downto 0);
        contador_extraunidades : out std_logic_vector(3 downto 0);
        contador_extradecenas : out std_logic_vector(3 downto 0);
        alarma            : out std_logic;
        felicitacion       : out std_logic
    );
end control_parqueo;

architecture comportamental of control_parqueo is
    signal tiempo_35s : integer range 0 to 35 := 0;
    signal tiempo_extra : integer range 0 to 99 := 0;
    type tipo_estado is (LIBRE, CONTANDO_35, EXTRA_TIEMPO, FIN_FELIZ);
    signal estado_actual : tipo_estado := LIBRE;
begin
    process(clk_1hz, rst)
    begin
        if rst = '1' then
            estado_actual <= LIBRE;
            tiempo_35s <= 0;
            tiempo_extra <= 0;
            alarma <= '0';
            felicitacion <= '0';
        elsif rising_edge(clk_1hz) then
            case estado_actual is
                when LIBRE =>
                    tiempo_35s <= 0;
                    tiempo_extra <= 0;
                    alarma <= '0';
                    felicitacion <= '0';
                    if sensor = '1' then
                        estado_actual <= CONTANDO_35;
                    end if;

                when CONTANDO_35 =>
                    if sensor = '0' then
                        estado_actual <= FIN_FELIZ;
                        felicitacion <= '1';
                    else
                        if tiempo_35s < 35 then
                            tiempo_35s <= tiempo_35s + 1;
                        else
                            estado_actual <= EXTRA_TIEMPO;
                        end if;
                    end if;
                end case;
            end case;
        end if;
    end process;
end architecture;
```

```

        alarma <= '1';
    end if;
end if;

when EXTRA_TIEMPO =>
    if sensor = '0' then
        estado_actual <= LIBRE;
    else
        if tiempo_extra < 99 then
            tiempo_extra <= tiempo_extra + 1;
        end if;
    end if;

when FIN_FELIZ =>
    if sensor = '1' then
        estado_actual <= CONTANDO_35;
        felicitacion <= '0';
        tiempo_35s <= 0;
    end if;
end case;
end if;
end process;

contador_35unidades    <= std_logic_vector(to_unsigned(tiempo_35s mod 10, 4));
contador_35decenas     <= std_logic_vector(to_unsigned(tiempo_35s / 10, 4));
contador_extraunidades <= std_logic_vector(to_unsigned(tiempo_extra mod 10, 4));
contador_extradecenas  <= std_logic_vector(to_unsigned(tiempo_extra / 10, 4));
end comportamental;

```

Para verificar que la integración es exitosa, ejecuta la herramienta "Analysis & Synthesis" en Quartus. Si la barra de progreso finaliza en verde sin advertencias de "Type Mismatch", los componentes se están comunicando adecuadamente.

*User prompt: corriji el error algo mas ?*

Response: Los cuatro archivos están correctamente estructurados y las incompatibilidades de tipo han sido resueltas. El archivo principal `sistema_de_parqueo_1` ahora se enlaza sin errores de síntesis con las salidas de 4 bits de `control_parqueo` y transfiere los datos a las instancias de `segundo_comp`. El módulo `divisor_1hz` también está bien instanciado para entregar la señal de reloj reducida.

Para ver el proyecto operando físicamente en tu tarjeta Altera DE0, debes ejecutar estos pasos finales en Intel Quartus Prime:

- **Asignación de Pines (Pin Planner):** El código es actualmente solo software. Ve a *Assignments > Pin Planner* y mapea cada puerto declarado en la entidad `sistema_de_parqueo_1` a los pines de la FPGA:
  - `CLOCK_50`: Conéctalo al pin físico del cristal de 50 MHz (usualmente `PIN_G21` o `PIN_B12` dependiendo de la versión exacta de la DE0).
  - `SW_SENSOR` y `SW_RESET`: Asígnalos a los interruptores deslizantes de la placa.
  - `LED_ALARMA` y `LED_FELIZ`: Asígnalos a dos pines de la barra de LEDs.

- HEX0, HEX1, HEX2, HEX3: Asigna cada uno de los 7 bits a los displays correspondientes de la tarjeta.
  - **Confirmación de Lógica en Displays:** Tu archivo `segundo_comp` asigna el valor "0000001" para formar el número cero. Esto indica que usas lógica activa en bajo (un '0' enciende el segmento y un '1' lo apaga). Esta configuración es la correcta y compatible con los displays de ánodo común integrados en la Altera DE0.
  - **Compilación y Programación:** Ejecuta *Compile Design* para compilar todo el hardware. Una vez generado el archivo `.sof` sin errores, abre la herramienta *Programmer*, detecta tu conexión USB-Blaster y carga el diseño a la placa.
-